

NATIONAL UNIVERSITY OF
MODERN LANGUAGES

Assignment # 02



SUBMITTED BY:
ZULQARNAIN HASSAN

SUBMITTED TO:
SIR. MUHAMMAD WAQAR

Subject: Data Structure and algorithms

Roll No: SP21379

Department: Software engineering

Section: BSSE-III-A-afternoon

Submission date.....25th may, 2023

Shell sort:

The shell sorting technique is based on the insertion sort. Sometimes we need to shift large blocks to insert items in the correct location. Using shell sort, we can avoid a large number of shifts. The sorting is done with specific intervals. After each pass the interval is reduced to make a smaller interval.

The interval between the elements is reduced based on the sequence used.

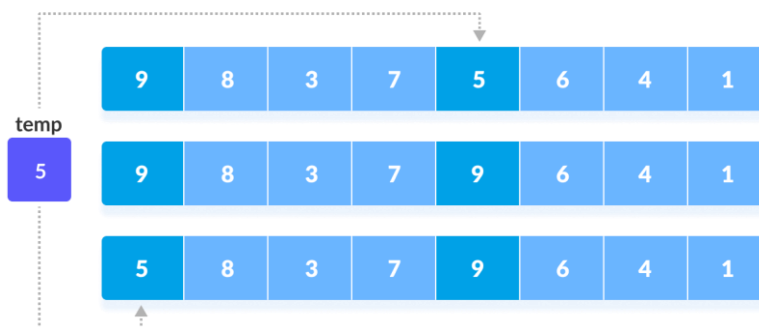
Shell's original sequence: $N/2, N/4, \dots, 1$.

Working of Shell Sort

Suppose, we need to sort the following array.



Rearrange all the elements at $n/2$ interval



5	8	3	7	9	6	4	1
5	6	3	7	9	8	4	1
5	6	3	7	9	8	4	1
5	6	3	1	9	8	4	7

Rearrange the elements at $n/4$ interval

5	6	3	1	9	8	4	7
3	6	5	1	9	8	4	7
3	1	5	6	9	8	4	7
3	1	5	6	9	8	4	7

3	1	5	6	9	8	4	7
---	---	---	---	---	---	---	---

3	1	5	6	9	8	4	7
---	---	---	---	---	---	---	---

3	1	4	6	5	8	9	7
---	---	---	---	---	---	---	---

3	1	4	6	5	8	9	7
---	---	---	---	---	---	---	---

3	1	4	6	5	7	9	8
---	---	---	---	---	---	---	---

Rearrange the elements at $n/8$ interval

3	1	4	6	5	7	9	8
1	3	4	6	5	7	9	8
1	3	4	6	5	7	9	8
1	3	4	6	5	7	9	8
1	3	4	5	6	7	9	8
1	3	4	5	6	7	9	8
1	3	4	5	6	7	9	8
1	3	4	5	6	7	9	8
1	3	4	5	6	7	8	9

Example:

```
#include <iostream>
```

```
using namespace std;
```

```
void shellSort(int array[], int n) {
```

```

for (int interval = n / 2; interval > 0; interval /= 2) {
    for (int i = interval; i < n; i += 1) {
        int temp = array[i];
        int j;
        for (j = i; j >= interval && array[j - interval] > temp; j -= interval) {
            array[j] = array[j - interval];
        }
        array[j] = temp;
    }
}
}

```

```

void printArray(int array[], int size) {
    int i;
    for (i = 0; i < size; i++)
        cout << array[i] << " ";
    cout << endl;
}

```

```

int main() {
    int data[] = {9, 8, 3, 7, 5, 6, 4, 1};
    cout<<"array without sorting:\n";
    int size = sizeof(data) / sizeof(data[0]);
    for(int i=0;i<size;i++)
    {
        cout<<data[i]<<" ";
    }
    shellSort(data, size);
    cout << "\nSorted array: \n";
    printArray(data, size);
}

```

}

```
PS E:\C++> cd "e:\C++\" ; if ($?) { g++ shellsort.cpp -o shellsort } ; if ($?) { .\shellsort }  
array without sorting:  
1 3 4 5 6 7 8 9  
Sorted array:  
1 3 4 5 6 7 8 9  
PS E:\C++>
```

Activate Wi

complexity

The time complexity of the code is $O(n^2)$ in the worst case.

The outer loop iterates $\log(n)$ times, where 'n' is the size of the array. The reason is that the interval is divided by 2 in each iteration until it becomes 0. This results in a logarithmic number of iterations.

The inner loop has a worst-case time complexity of $O(n)$ because it iterates over each element of the array once.

Therefore, the overall time complexity of the shell sort algorithm implemented in the given code is $O(n^2)$, where 'n' is the size of the array.